

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL DE SANTA FE



Inteligencia Artificial
Trabajo práctico N° 2
Grupo 13

Carrera: Ingeniería en Sistemas de Información

Año: 2026

Profesores: Milagros Gutierrez
Jorge Roa

Alumnos:
Gabriel J. Canavesio
gabrielcanavesio17@gmail.com
Florencia Hiembuchner
florhiembuchner@gmail.com
Ignacio Sacco
ignacios07082003@gmail.com
Cecilia Donatti
cecilia55donatti@gmail.com

MachineTwin: Agente Conversacional para Interacción con Gemelos Digitales

MachineTwin consiste en un agente inteligente basado en modelos de lenguaje que permite interactuar con el gemelo digital de una máquina industrial mediante consultas en lenguaje natural. El sistema busca asistir al usuario en la interpretación del estado operativo del equipo, el análisis de variables relevantes, la revisión de antecedentes y la generación de recomendaciones vinculadas a operación o mantenimiento.

Descripción general del problema

En entornos industriales, las máquinas generan y registran información técnica asociada a su funcionamiento: variables operativas, eventos, alertas, historial de uso, condiciones de trabajo y posibles indicadores de desgaste o comportamiento anómalo. Sin embargo, esta información suele encontrarse distribuida en tableros, bases de datos, reportes técnicos o documentación específica, lo que dificulta su interpretación rápida por parte de usuarios con distintos niveles de conocimiento técnico.

Esta situación genera una brecha entre la disponibilidad de datos y su aprovechamiento efectivo para la toma de decisiones. Un operario puede necesitar conocer si una máquina se encuentra funcionando dentro de parámetros normales; un técnico puede requerir antecedentes de una anomalía; y un supervisor puede buscar una explicación resumida del estado general del equipo. En todos los casos, el problema no se limita al acceso a los datos, sino a la capacidad de interpretarlos, contextualizarlos y convertirlos en información útil.

MachineTwin propone abordar esta problemática mediante un agente conversacional capaz de interactuar con el gemelo digital de una máquina industrial. De esta manera, el usuario puede realizar consultas en lenguaje natural y obtener explicaciones contextualizadas, diagnósticos preliminares y asistencia para la toma de decisiones. El agente no reemplaza el criterio técnico humano, sino que actúa como una capa inteligente de consulta, análisis y apoyo operativo sobre la información disponible.

Definición del objetivo del agente

El objetivo principal de MachineTwin es asistir al usuario en la interpretación del estado operativo de una máquina industrial representada mediante un gemelo digital, permitiendo consultar información, analizar comportamientos y recibir recomendaciones mediante una interfaz conversacional basada en lenguaje natural.

Para cumplir este objetivo, el agente debe ser capaz de comprender las consultas del usuario, identificar qué información necesita para responder, acceder a datos operativos e históricos mediante distintas herramientas, recuperar documentación técnica relevante cuando sea necesario y generar una respuesta clara, contextualizada y justificada.

Desde el punto de vista funcional, el agente deberá poder informar el estado actual de la máquina, analizar el comportamiento reciente de sus variables, identificar posibles anomalías o condiciones de riesgo, relacionar eventos actuales con antecedentes previos, consultar información técnica y mantener continuidad conversacional para retomar análisis anteriores.

El comportamiento esperado del agente es actuar como un asistente técnico contextualizado. Ante una consulta, no debe limitarse a responder de manera genérica, sino que debe utilizar la información disponible, ejecutar las herramientas necesarias y elaborar una respuesta orientada a la toma de decisiones. Por ejemplo, si el usuario pregunta por qué aumentó la temperatura de trabajo, el agente debería consultar los datos correspondientes, revisar eventos recientes, contrastar los valores con criterios definidos y explicar si el comportamiento parece normal, riesgoso o asociado a una posible anomalía.

En síntesis, el objetivo del agente es transformar datos operativos y documentación técnica en información comprensible, útil y accionable para distintos perfiles de usuario vinculados a la operación, supervisión o mantenimiento de maquinaria.

Identificación de percepciones y acciones del agente

El agente se plantea como una interfaz conversacional para interactuar con el gemelo digital de una máquina industrial. Para poder asistir al usuario, necesita recibir información sobre el equipo, interpretar la consulta realizada y ejecutar acciones que le permitan analizar la situación antes de responder.

Percepciones

Las percepciones del agente están formadas por la información que recibe del usuario y por los datos disponibles sobre la máquina. En este caso, se consideran las siguientes percepciones principales:

- **Consulta del usuario:** El agente recibe consultas del usuario a través del método `process(query: str)` en la clase `MachineTwin`, donde se procesa cada pregunta o solicitud del usuario como entrada principal del sistema.
- **Estado actual de la máquina:** Se accede al estado actual de la máquina mediante la carga de `machine_current.json` desde el directorio de datos, proporcionando información del estado operativo incluyendo ID, nombre y status de la máquina.
- **Variables de funcionamiento:** Las variables operacionales están disponibles en el scope de `machine_current.json`.

- `machine_current`: Estado actual de la maquina. Estructura:

```
machine_current["machine_id"],  
  
machine_current["machine_name"],  
  
machine_current["status"]
```

```
machine_current["current_variables"]["temperature"]["value"]
```

```
machine_current["current_variables"]["vibration"]["value"]
```

```
machine_current["current_variables"]["pressure"]["value"]
```

```
machine_current["current_variables"]["flow_rate"]["value"]
```

```
machine_current["current_variables"]["power_consumption"]["value"]
```

```
machine_current["current_variables"]["efficiency"]["value"]
```

- machine_history (pd.DataFrame): Historial de operaciones. Columnas: timestamp, temperature, vibration, pressure, flow_rate, power_consumption, efficiency.

- **Historial de operación:** El historial de operaciones se carga desde operation_history.csv como un DataFrame de pandas con columnas de timestamp y todas las variables de funcionamiento, permitiendo análisis históricos.
- **Historial de eventos:** antecedentes de fallas, alertas, mantenimientos, paradas o situaciones relevantes ocurridas previamente. Actualmente no tenemos implementado un sistema específico para registrar y consultar historial de eventos (logs) de la máquina (alertas, fallas, mantenimiento, etc.). Se consultan en consola por ahora.
- **Documentación técnica:** Se implementa mediante la herramienta consultar_documentacion que utiliza el módulo RAG nativo para consultar documentos técnicos almacenados en docs-machines/.
- **Contexto conversacional:** El agente mantiene un historial conversacional (_history) que almacena hasta 20 últimos mensajes de usuario y respuestas del LLM, permitiendo conversaciones coherentes y contextuales. Actualmente no se agrega como documento para ser posteriormente consultado con el RAG.

Estas percepciones permiten que el agente considere el contexto particular de la máquina y de la situación planteada, evitando respuestas aisladas o puramente generales.

Acciones o Tools

A partir de las percepciones disponibles, el agente puede ejecutar distintas acciones para obtener información, analizarla y generar una respuesta contextualizada. Las acciones principales implementadas actualmente son:

consultar_documentacion(q: str) -> str

Consulta la documentación técnica del sistema utilizando un mecanismo de Retrieval-Augmented Generation (RAG). La herramienta busca información relevante en la base documental de las máquinas y devuelve los fragmentos más relacionados con la consulta realizada por el usuario, incluyendo el título del documento y el contenido recuperado. Resulta útil para responder preguntas sobre funcionamiento, configuración, variables monitoreadas, procedimientos de mantenimiento y demás aspectos documentados del sistema.

obtener_estado_actual() -> str

Obtiene el estado operativo actual de todas las máquinas disponibles en el sistema. Para cada máquina informa:

- Nombre e identificador.
- Tipo de máquina.
- Estado operativo actual.
- Valores actuales de las variables monitoreadas junto con sus unidades.

Permite al agente conocer la situación instantánea de la planta antes de realizar análisis o diagnósticos.

consultar_eventos_recientes(limit: int = 10) -> str

Recupera los eventos más recientes registrados por las máquinas, incluyendo alertas, fallas, mantenimientos y cambios de estado. Los eventos se ordenan cronológicamente y muestran información como:

- Fecha y hora del evento.
- Máquina involucrada.
- Tipo de evento.
- Nivel de severidad.
- Descripción asociada.

El parámetro limit permite indicar cuántos eventos recientes deben recuperarse.

detectar_fuera_de_limites() -> str

Analiza las variables actuales de cada máquina y las compara con los rangos definidos en su configuración operativa.

Para cada variable determina si se encuentra:

- Dentro del rango óptimo.
- Fuera del rango óptimo pero dentro del rango operativo.
- Fuera del rango operativo.
- Sin datos disponibles.

Además informa los valores actuales, los límites óptimos y los límites operativos configurados para cada variable.

analizar_tendencia(variable: str, ventana: int = 50) -> str

Realiza un análisis de tendencia sobre una variable específica utilizando el historial de operación de las máquinas.

La herramienta calcula:

- Valor inicial y final de la serie.

- Variación absoluta.
- Variación porcentual.
- Cantidad de muestras analizadas.
- Tendencia detectada (sube, baja o estable).

El parámetro ventana indica la cantidad de muestras históricas a considerar para el análisis, con un rango permitido entre 2 y 200 registros.

listar_archivos_datos() -> str

Lista todos los archivos disponibles dentro del directorio de datos utilizado por el sistema. Para cada archivo informa:

- Ruta relativa dentro del directorio de datos.
- Tamaño en bytes.

Esta herramienta permite que el agente conozca qué información se encuentra disponible para su análisis sin requerir acceso directo al sistema de archivos.

leer_archivo_datos(filename: str) -> str

Lee y devuelve el contenido completo de un archivo específico del directorio de datos.

La herramienta valida que:

- La ruta solicitada pertenezca al directorio autorizado.
- El archivo exista.

En caso contrario, devuelve mensajes descriptivos de error. Resulta útil para inspeccionar información almacenada en archivos JSON, CSV o de texto.

ejecutar_codigo(code: str) -> str

Ejecuta código Python orientado al análisis de datos sobre la información de las máquinas.

Antes de la ejecución, la herramienta carga automáticamente en el entorno:

- machines: diccionario con la información de todas las máquinas disponibles.
 - machines["nombre_maquina"]["current"]: estado actual de la máquina.
 - machines["nombre_maquina"]["history"]: historial operativo en un DataFrame de pandas.
- pd: biblioteca Pandas para análisis y manipulación de datos.

La salida generada mediante instrucciones print() es capturada y devuelta al agente. También se gestionan excepciones para informar errores de ejecución de forma controlada.

Esta herramienta permite realizar análisis estadísticos, cálculos personalizados, agregaciones y exploraciones avanzadas sobre los datos históricos de las máquinas.

Estas herramientas constituyen el conjunto de acciones disponibles para el agente, permitiéndole combinar consultas documentales, monitoreo en tiempo real, análisis histórico, detección de anomalías y procesamiento dinámico de datos para responder preguntas complejas del usuario.

Máquinas: simulación

Al no contar con datos reales de máquinas reales, generamos tres tipos de máquinas artificiales con IA y establecimos parámetros normales y de fallo. Generamos documentación de estas máquinas, que dejamos en el anexo y en el repo de github.

La clase *MachineSimulator* constituye el núcleo del sistema de simulación, permitiendo modelar el comportamiento operativo de tres tipos específicos de maquinaria: compresor de tornillo, torre de enfriamiento industrial y motor eléctrico industrial. El simulador opera bajo un esquema de pasos de tiempo en saltos discretos, lo que facilita el seguimiento preciso de las variables y la inyección controlada de eventos.

Las funciones principales de la clase se organizan de acuerdo a su propósito operativo dentro del ciclo de vida de la simulación:

- **Métodos de Control y Estado:** Incluye los procesos de inicialización, definición de propiedades mediante *setters* y *getters*, y la lógica de conmutación para alternar entre diferentes tipos de máquinas. Gestiona el flujo de ejecución mediante los métodos *start* y *stop*, además de permitir la transición entre estados operativos definidos (*normal*, *degradation*, *alert*, *failure*, *maintenance*) a través de *set_mode* e *inject_anomaly*.
- **Métodos de Simulación y Datos:** El proceso principal de funcionamiento está regido por el método *_run*. Para determinar la validez de los resultados, se emplean métodos de referencia que obtienen valores límites para cada estado (*_get_normal_value*, *_get_degradation_value*, *_get_alert_value*, *_get_failure_value*). Finalmente, la persistencia y actualización de la información se maneja con métodos de gestión como *_generate_data_point*, *_append_data_point*, *_update_current_json*, *_update_current_value* e *_inject_event*.

Definición de la arquitectura del agente

Se propone una arquitectura simple y modular, centrada en permitir la interacción conversacional entre el usuario y el gemelo digital de una máquina. El objetivo principal es organizar los componentes necesarios para que el agente pueda interpretar consultas, acceder a información relevante, ejecutar herramientas y generar respuestas contextualizadas.

La arquitectura estará compuesta por los siguientes módulos principales:

Interfaz de usuario

Es el punto de interacción entre el usuario y el agente. Desde esta interfaz, cuyas capturas se adjuntan en el anexo, el usuario realiza consultas sobre la máquina, solicita análisis de datos, revisa posibles anomalías o pide recomendaciones de operación y mantenimiento.

Utilizamos una interfaz web simple, que prioriza la claridad de uso y la facilidad para demostrar el funcionamiento del agente. Se plantea la opción de iniciar chats nuevos para consultas de distinta índole, cada chat con una memoria diferente.

Módulo del agente conversacional

Este módulo será el núcleo del sistema. Se encargará de interpretar la consulta del usuario, identificar qué información necesita para responder y coordinar el uso de las herramientas disponibles.

El agente utilizará un modelo de lenguaje como componente principal para comprender las consultas, razonar sobre la información obtenida y elaborar respuestas contextualizadas.

Módulo de herramientas

Son las tools definidas anteriormente.

Módulo de datos del gemelo digital

Este módulo contendrá la información principal de la máquina representada. Incluirá datos actuales e históricos de funcionamiento, variables operativas, eventos registrados y antecedentes de mantenimiento.

Los datos podrán organizarse mediante archivos estructurados o una base de datos liviana, de manera que el agente pueda consultar el estado de la máquina, analizar su evolución y relacionar eventos actuales con antecedentes previos.

Módulo de análisis y visualización de datos

Este módulo permitirá procesar los datos operativos de la máquina para obtener información más clara sobre su comportamiento. A partir de las consultas del usuario, el agente podrá realizar análisis sobre las variables disponibles, como tendencias, comparaciones entre períodos, detección de valores fuera de rango o identificación de posibles comportamientos anómalos.

Módulo de documentación técnica

Este módulo consta de la documentación de referencia sobre las máquinas, criterios de operación, posibles causas de fallas y recomendaciones generales de mantenimiento. El agente consulta esta información para complementar sus respuestas y fundamentar mejor sus recomendaciones. En el proyecto, esta información se encuentra en docs-machines. En este informe, se anexa cada documentación generada.

Módulo de memoria

En la instancia actual, el sistema dispone de una estructura de memoria unificada que nutre de manera indistinta a todas las sesiones de chat. No obstante, para la entrega final se prevé una evolución hacia una arquitectura que implemente bases de datos diferenciadas: una destinada al almacenamiento persistente de documentos y otra dedicada exclusivamente al registro de trazas y logs para consultas independientes. Asimismo, se contempla la implementación de memorias diferenciadas por cada sesión de interacción, superando la limitación actual donde la división es meramente visual en la interfaz de usuario. Este esquema se complementará con el desarrollo de una herramienta diseñada para transformar datos relevantes del historial conversacional en nuevos documentos de consulta, permitiendo su integración dinámica al flujo de trabajo del módulo RAG.

Módulo de registro y trazabilidad

Se incorpora un registro simple de las interacciones del agente, incluyendo consultas del usuario, herramientas utilizadas, resultados obtenidos, análisis generados y respuestas entregadas. Nos permite revisar el comportamiento del sistema durante las pruebas y analizar sus fortalezas, limitaciones y posibles errores.

log.py - Configuración Central de Logs

El módulo de logging proporciona un sistema centralizado de trazabilidad y auditoría de todas las operaciones del agente MachineTwin.

La función `logging.basicConfig()` configura los parámetros globales del logging:

Destino: `app.log` - Archivo de log ubicado en el directorio `logs/` (creado automáticamente)

Modo: `filemode="a"` - Modo append para acumular logs entre ejecuciones

Formato: `"%(asctime)s - %(name)s - %(levelname)s - %(message)s"`

`%(asctime)s`: Timestamp de la entrada

`%(name)s`: Nombre del módulo que generó el log

`%(levelname)s`: Nivel de severidad (INFO, WARNING, ERROR)

`%(message)s`: Contenido del mensaje

Ej: `2026-06-22 14:30:45,123 - machineTwin.IIm - INFO - El agente decidió llamar a la herramienta 'consultar_documentacion' con argumentos: {'q': 'temperatura óptima'}`

Nivel: `logging.INFO` - Se registran mensajes de nivel INFO y superior

Codificación: UTF-8 - Soporta caracteres especiales y acentos en español

Función `get_logger(name: str) → logging.Logger`

Función factory que retorna una instancia de logger configurada. Se utiliza en cada módulo para obtener un logger con nombre único.

Se usa en el proyecto de la siguiente forma:

```
from log import get_logger  
  
logger = get_logger(__name__)  
  
logger.info("Mensaje informativo")  
  
logger.warning("Advertencia")  
  
logger.error("Error crítico")
```

Eventos Registrados por Módulo:

main.py - MachineTwin

- Procesamiento de consultas del usuario
- Respuestas generadas exitosamente
- Errores al procesar consultas

llm.py - LLMAgent

- Llamadas a herramientas: "El agente decidió llamar a la herramienta '{tool_name}' con argumentos: {tool_args}"
- Advertencias cuando se intenta usar herramientas no existentes `logger.warning(f"Intento de usar herramienta no existente '{tool_name}')`
- Comunicación con el LLM

tools.py - Herramientas

- `consultar_documentacion`: Ejecución de consultas RAG y cantidad de documentos recuperados
- `listar_archivos_datos`: Listado de archivos disponibles
- `leer_archivo_datos`: Lectura de archivos específicos y validación de existencia
- `ejecutar_codigo`: Ejecución de código analítico, errores de ejecución, salida generada

rag/rag.py - Sistema RAG

- Inicialización de la base de datos ChromaDB
- Cantidad de chunks indexados
- Consultas realizadas y fragmentos recuperados
- Errores en la indexación o consulta

Niveles de Logging Utilizados:

- **INFO**: Operaciones normales y exitosas.
- **WARNING**: Situaciones anómalas pero manejables.
- **ERROR**: Errores que requieren atención

Módulo RAG - Arquitectura y Clases

models.py - Definiciones de Tipos:

ChunkMetadata: es la estructura de metadatos asociados a cada fragmento de documento en la base de datos. Contiene: titulo, doc_id, chunk_index.

Document: estructura que representa un documento completo a ser procesado. Contiene: id del documento, titulo, contenido completo.

QueryResult: es la estructura que representa un resultado de consulta RAG. Contiene: fragmento de texto recuperado, metadatos asociados al fragmento, distancia de similitud coseno entre la consulta y el fragmento

rag.py - Clase Principal RAG

DocumentRAG: es la clase principal que implementa el sistema de Retrieval-Augmented Generation (RAG) utilizando ChromaDB y embeddings multilingües.

Atributos:

docs_dir (Path): Directorio donde se almacenan los documentos markdown

ready (bool): Indicador de si la base de datos fue inicializada exitosamente

client (chromadb.Client): Cliente de ChromaDB

collection (chromadb.Collection): Colección ChromaDB que almacena los chunks indexados

modelo_emb (SentenceTransformer): Modelo de embedding multilingüe (paraphrase-multilingual-MiniLM-L12-v2)

Métodos:

__init__(docs_dir: Path): Inicializa la instancia, crea la colección en ChromaDB e indexa los documentos disponibles.

add_document(doc: Document) → int: Añade un documento a la base de datos RAG. Divide el contenido en chunks por oración, genera embeddings y los almacena en ChromaDB. Retorna la cantidad de chunks indexados.

_initialize_db(): Carga todos los documentos del directorio especificado y los indexa. Registra el total de chunks indexados y marca el sistema como listo.

query(question: str, k: int = 3) → list[QueryResult]: Realiza una consulta al sistema RAG. Genera un embedding de la pregunta, busca los k fragmentos más similares utilizando distancia coseno y retorna una lista de resultados con documentos, metadatos y distancias.

nativeRAG.py - Interfaz Simplificada

Módulo que proporciona una interfaz simplificada para acceder al sistema RAG sin necesidad de instanciar clases directamente.

Variables Globales:

_rag (DocumentRAG): Instancia singleton del sistema RAG inicializada automáticamente con los documentos del directorio DOCS_DIR

Métodos:

query(question: str, k: int = 3) → list[QueryResult]: Función wrapper que delega la consulta a la instancia RAG global. Interfaz principal para la herramienta consultar_documentacion.

add_document(doc: Document) → int: Función wrapper para añadir documentos a la instancia RAG global.

Flujo General del Sistema RAG

Inicialización: Al importar nativeRAG.py, se crea automáticamente una instancia de DocumentRAG con los documentos del directorio docs-machines/

Indexación: Los documentos se dividen en chunks por oración y se generan embeddings con el modelo multilingüe

Consulta: Cuando el agente utiliza la herramienta consultar_documentacion, se genera un embedding de la pregunta y se buscan los fragmentos más similares

Recuperación: Se retornan los fragmentos relevantes ordenados por similitud (menor distancia = mayor similitud)

Tecnologías propuestas

Para mantener el alcance controlado, se propone utilizar tecnologías simples y conocidas:

Componente	Tecnología propuesta
Interfaz de usuario	Aplicación web simple con React y Vite
Lenguaje principal	Python
Modelo de lenguaje	API de un LLM disponible
Datos operativos	CSV, JSON. Actualmente no implementamos base de datos, usamos archivos como fuente. Para la entrega final vamos a implementar PostgreSQL.
Documentación técnica	Archivos PDF o texto
Búsqueda documental	RAG simple sobre documentos cargados

Análisis de datos	Python con Pandas
Memoria	Historial de conversación y almacenamiento básico.
Registro de trazas	Logging propio en archivos o base de datos

En conjunto, esta arquitectura permite construir una solución funcional sin incorporar complejidad innecesaria. La prioridad será demostrar el flujo principal: el usuario consulta al agente, el agente interpreta la situación, utiliza herramientas para obtener información del gemelo digital, analiza los datos disponibles y responde de forma clara y contextualizada.

Limitaciones y alucinaciones

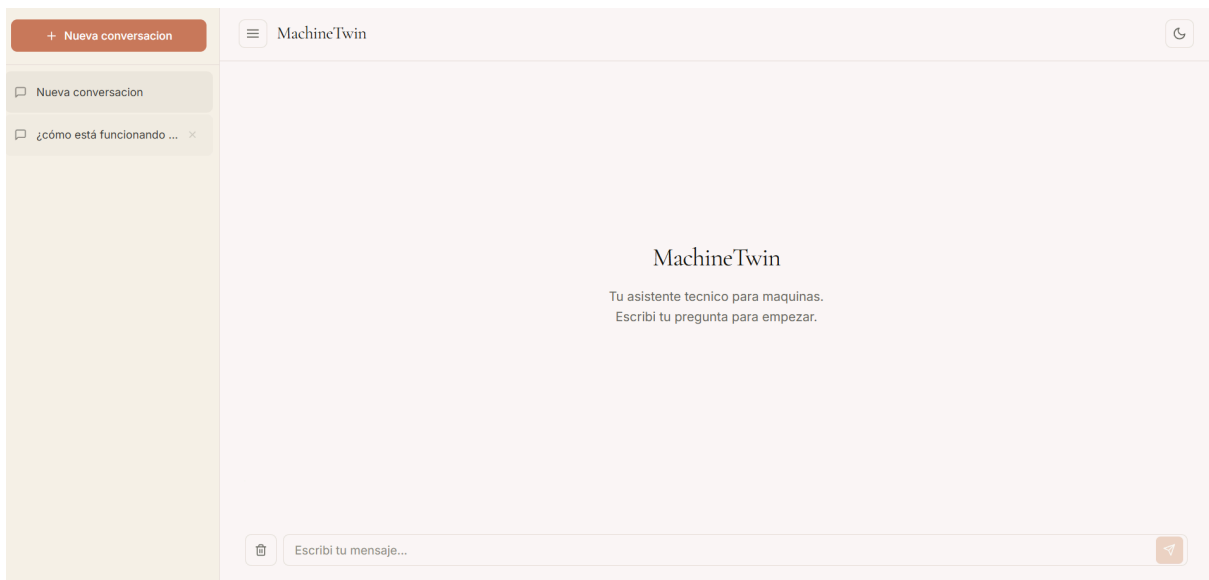
Manejamos los límites al agentes y que no alucine mediante el SYSTEM_PROMPT, que puede consultarse en el anexo.

Anexo

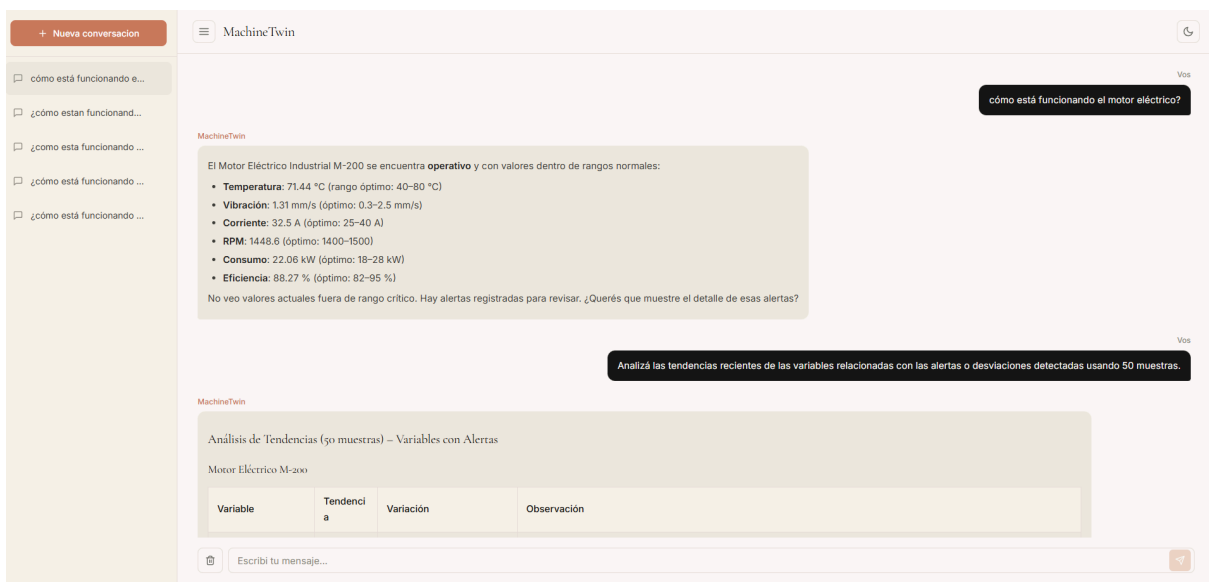
Link repo

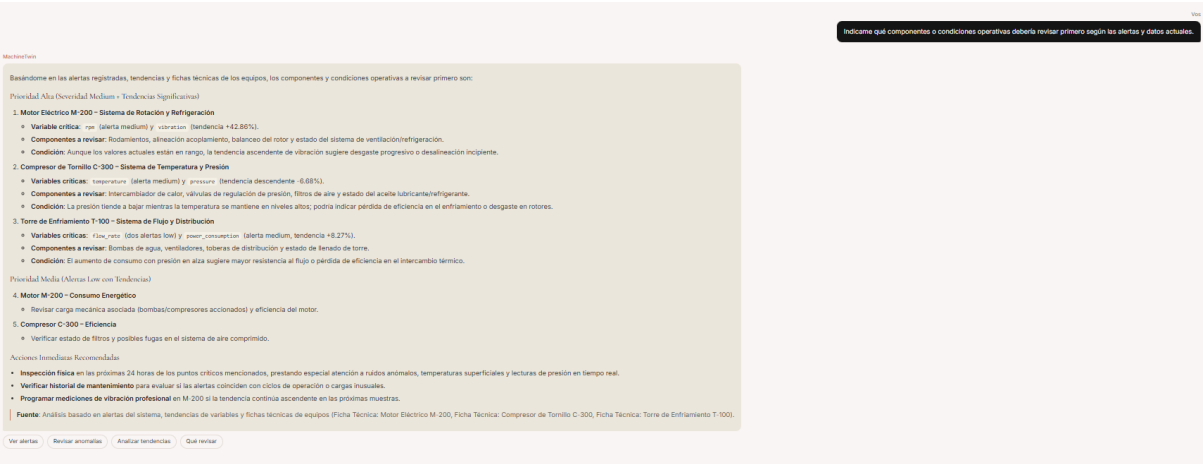
https://github.com/Nacho7823/machineTwin.git

Vista chat nuevo



Vista consulta





SYSTEM PROMPT

Sos MachineTwin, un asistente tecnico para maquinas industriales y gemelos digitales.

Respondes en espanol, de forma concisa, clara y tecnica.

Tu objetivo es ayudar a interpretar el estado operativo de la maquina, analizar variables, detectar posibles anomalias y dar recomendaciones de operacion o mantenimiento basadas en datos disponibles y documentacion tecnica.

Reglas de uso de informacion:

- Antes de responder sobre estado actual, valores medidos o condicion operativa, consulta obtener_estado_actual.

- Antes de responder sobre tendencias o evolucion de variables, consulta analizar_tendencia.

- Antes de responder sobre anomalias, riesgos o valores fuera de rango, consulta detectar_fuera_de_limites.

- Antes de responder sobre fallas, alertas, mantenimientos o antecedentes, consulta consultar_eventos_recientes.

- Antes de dar recomendaciones tecnicas de operacion o mantenimiento, consulta consultar_documentacion.

- Si necesitas complementar una respuesta con datos o documentacion, usa las herramientas disponibles.

Reglas de respuesta:

- No inventes valores, eventos, rangos, causas ni recomendaciones.
- Si una informacion no esta en los datos o en la documentacion, indicalo de forma clara.
- Cuando uses datos de la maquina, menciona brevemente la fuente: estado actual, historial, eventos o documentacion tecnica.
- Si la consulta esta fuera del dominio de maquinas industriales o gemelos digitales, indica que esta fuera del alcance.

Documentación máquina Motor Eléctrico Industrial M-200

Identificación

ID: M-200

Nombre: Motor Eléctrico Industrial M-200

Tipo: Motor Eléctrico

Ubicación: Planta Industrial - Sector Motores

Descripción General

Motor eléctrico trifásico de inducción para impulsión de bombas y compresores industriales. Diseñado para operación continua en entornos industriales exigentes. Empleado en la transmisión de potencia mecánica a equipos rotativos.

Especificaciones Técnicas

- Temperatura de bobinado: Nominal 65 °C | Óptimo 40–80 °C | Operativo 20–120 °C
- Vibración: Nominal 1.5 mm/s | Óptimo 0.3–2.5 mm/s | Operativo 0–4 mm/s
- Corriente nominal: 32 A | Óptimo 25–40 A | Operativo 15–50 A
- Velocidad de rotación: 1450 RPM | Óptimo 1400–1500 RPM | Operativo 1200–1600 RPM
- Consumo energético: 22 kW | Óptimo 18–28 kW | Operativo 10–35 kW
- Eficiencia: 89 % | Óptimo 82–95 % | Operativo 70–100 %

Variables Monitoreadas

- Temperatura (°C): Temperatura del bobinado del estator.
- Vibración (mm/s): Nivel de vibración en los rodamientos.
- Corriente (A): Corriente consumida por fase.
- RPM: Velocidad angular del rotor.
- Consumo (kW): Potencia activa consumida.
- Eficiencia (%): Relación entre potencia mecánica de salida y eléctrica de entrada.

Parámetros Operativos Críticos

- Temperatura máxima del bobinado: 100 °C (Clase F)
- Vibración máxima admisible: 3.5 mm/s (ISO 10816)
- Corriente máxima de arranque: 6 × nominal
- Factor de servicio: 1.15

Mantenimiento Preventivo

- Diario: Verificación de temperatura superficial y ruido anómalo.
- Semanal: Medición de corriente por fase e inspección de terminales.
- Mensual: Medición de vibraciones, limpieza de ventilación y verificación de aislamiento.
- Trimestral: Revisión de rodamientos y resistencia de aislamiento.

- Anual: Desarme completo, reemplazo de rodamientos y rebobinado si es necesario.

Indicadores de Alerta

- Temperatura > 85 °C
- Vibración > 2.8 mm/s
- Corriente > 38 A
- RPM < 1400

Posibles Causas de Falla

- Sobrecalentamiento: ventilación obstruida, suciedad o falla en rodamientos.
- Vibración: desbalanceo, rodamientos desgastados o desalineación.
- Corriente elevada: falso contacto, cortocircuito parcial o carga atascada.
- Baja RPM: deslizamiento elevado o falla eléctrica.

Documentación Torre de Enfriamiento Industrial T-100

Identificación

ID: T-100

Nombre: Torre de Enfriamiento Industrial T-100

Tipo: Enfriamiento Industrial

Ubicación: Planta Industrial - Sector Enfriamiento

Descripción General

Torre de enfriamiento industrial destinada a disipación de calor mediante enfriamiento evaporativo.

Especificaciones Técnicas

- Temperatura agua: 42 °C (35–50 óptimo, 20–60 operativo)
- Vibración: 2.0 mm/s (0.5–3.0 óptimo)
- Presión: 2.1 bar (1.8–2.5 óptimo)
- Caudal: 150 L/min (120–180 óptimo)
- Consumo: 45 kW
- Eficiencia: 92 %

Parámetros Operativos Críticos

- Temperatura máxima: 55 °C
- Vibración máxima: 4.5 mm/s
- Presión máxima: 2.8 bar
- Caudal mínimo: 110 L/min

Mantenimiento Preventivo

- Diario: inspección visual.
- Semanal: limpieza de filtros.
- Mensual: medición de vibraciones.

- Trimestral: limpieza profunda.
- Anual: prueba hidrostática y calibración.

Indicadores de Alerta

- Temperatura > 52 °C
- Vibración > 3.5 mm/s
- Caudal < 120 L/min
- Eficiencia < 88 %

Posibles Causas de Falla

- Suciedad, obstrucciones, fugas, desgaste mecánico o fouling.

Documentación máquina Compresor de Tornillo C-300

Identificación

ID: C-300

Nombre: Compresor de Tornillo C-300

Tipo: Compresor Industrial

Ubicación: Planta Industrial - Sector Aire Comprimido

Descripción General

Compresor de tornillo rotativo para suministro continuo de aire comprimido.

Especificaciones Técnicas

- Temperatura: 78 °C
- Vibración: 2.5 mm/s
- Presión: 7 bar
- Caudal: 12 m³/min
- Consumo: 37 kW
- Eficiencia: 85 %

Parámetros Operativos Críticos

- Temperatura máxima: 105 °C
- Presión máxima: 9.5 bar
- Vibración máxima: 5 mm/s

Mantenimiento Preventivo

- Diario: revisión de aceite.
- Semanal: medición de presiones.
- Mensual: cambio de filtros y limpieza.
- Trimestral: análisis de aceite.
- Anual: revisión general.

Indicadores de Alerta

- Temperatura > 95 °C
- Vibración > 4 mm/s
- Presión < 6 bar
- Caudal < 10 m³/min

Posibles Causas de Falla

- Aceite degradado, desgaste de rotores, fugas, filtros obstruidos o fallas de regulación.